

Containerization with Docker

Mastering Docker

Table of Contents

Activity 1: Docker Installation	4
1.1 Prerequisite	4
1.2 Scenario	4
1.3 Steps	4
Activity 2: Create Docker File	9
2.1 Prerequisite	9
2.2 Scenario	9
2.3 Steps	9
Activity 3: Create DockerHub Account	10
3.1 Prerequisite	10
3.2 Scenario	10
3.3 Steps	10
Activity 4: Create Docker Image	11
4.1 Prerequisite	11
4.2 Scenario	11
4.3 Steps	11
Activity 5: Tag, List, Delete Image	14
5.1 Prerequisite	14
5.2 Scenario	15
5.3 Steps	15
Activity 6: Push Image to dockerhub Registry	16
6.1 Prerequisite	16
6.2 Scenario	16
6.3 Steps	16
Activity 7: Pull Image from dockerhub Registry	18
7.1 Prerequisite	18
7.2 Scenario	18
7.3 Steps	18
Activity 8: Create Container	19
8.1 Prerequisite	19
8.2 Scenario	19
8.3 Steps	19
Activity 9: Stop, Pause, List, Remove Container	20
9.1 Prerequisite	20

9.2	Scenario	20
9.3	Steps	20
Activity 10: Create MySQL Image		21
10.1	Prerequisite	21
10.2	Scenario	21
10.3	Steps.....	21
Activity 11: Create and Attach Network		26
11.1	Prerequisite	26
11.2	Scenario	26
11.3	Steps.....	26
Activity 12: Create and Mount Volume		28
12.1	Prerequisite	28
12.2	Scenario	29
12.3	Steps.....	29
Activity 13: Run Spring Boot and MySQL using Docker Compose		31
13.1	Prerequisite	31
13.2	Scenario	31
13.3	Steps.....	31

Activity 1: Docker Installation

1.1 Prerequisite

AWS cloud account is ready to use.

1.2 Scenario

Launch AWS EC2 Instance Connect to install docker.

1.3 Steps

1

Open AWS cloud console-> EC2-> instances-> Search instance by name or id

EC2 > Instances

EC2

Dashboard

AWS Global View

Events

Instances

Instances (246)

DevOps-Ubuntu

Use: "DevOps-Ubantu"

Tags values

Name = devops-ubuntu

4e436

Running

t2.large

2/2 checks passed

View alarms

1c8c0

Stopped

t2.xlarge

-

View alarms

DevOps-Ubantu

Clear filters

1

	Name	Instance ID	Instance state	Instance type	Status check	Alarm status
☐	devops-ubuntu	i-00fa815e45aaaf0d2	Running	t2.large	2/2 checks passed	View alarms

2

Click the **Connect** button to view the different options for connecting to your AWS instance.

EC2 > Instances > i-00fa815e45aaaf0d2

EC2

Dashboard

AWS Global View

Events

Instances

Instances

Instance Types

Launch Templates

Spot Requests

Savings Plans

Reserved Instances

Dedicated Hosts

Capacity Reservations

Capacity Manager

Instance summary for i-00fa815e45aaaf0d2 (devops-ubuntu)

Connect

Instance state

Actions

Updated less than a minute ago

Instance ID

i-00fa815e45aaaf0d2

IPv6 address

-

Hostname type

IP name: ip-172-31-68-166.ap-south-1.compute.internal

Answer private resource DNS name

-

Public IPv4 address

3.110.156.131 | open address

Instance state

Running

Private IP DNS name (IPv4 only)

ip-172-31-68-166.ap-south-1.compute.internal

Instance type

t2.large

Private IPv4 addresses

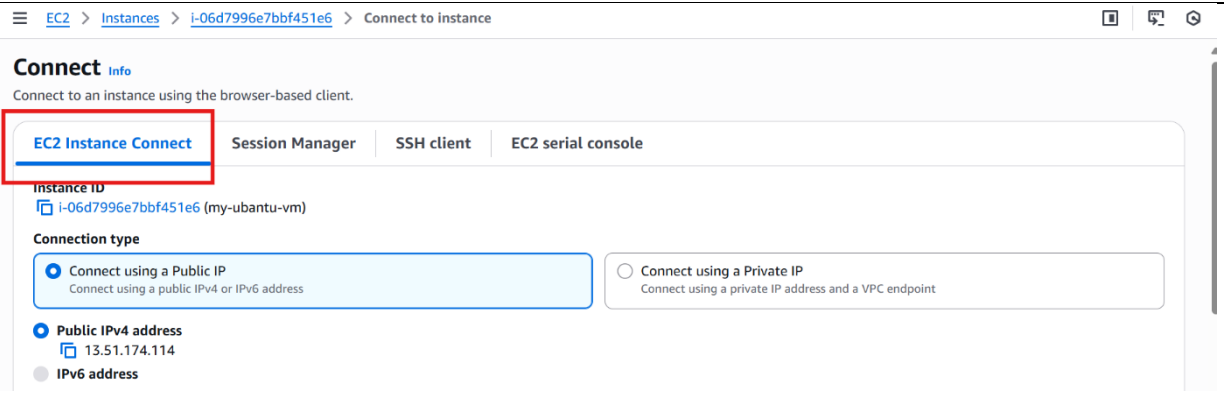
172.31.68.166

Public DNS

ec2-3-110-156-131.ap-south-1.compute.amazonaws.com | open address

Elastic IP addresses

-

	 Note: If you are using the Ubuntu Desktop (GUI) version, open a Terminal window and execute the commands listed below.
3	Uninstall any old docker version: Execute below command: <pre>sudo apt-get remove -y docker docker-engine docker.io containerd runc</pre>
4	Execute below command to install required dependencies sudo apt-get update -y <pre>ubuntu@ip-172-31-23-0:~\$ sudo apt-get update -y Hit:1 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble InRelease Get:2 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB] Get:3 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB] Get:4 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB] Get:5 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1581 kB] Get:6 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main Translation-en [298 kB] Get:7 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [175 kB] Get:8 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble-updates/universe amd64 Components [378 kB] Get:9 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble-updates/restricted amd64 Packages [2240 kB] Get:10 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble-updates/restricted Translation-en [508 kB] Get:11 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble-updates/restricted amd64 Components [212 B] Get:12 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 Components [940 B] Get:13 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble-backports/main amd64 Components [7132 B] Get:14 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble-backports/universe amd64 Components [11.0 kB] Get:15 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble-backports/restricted amd64 Components [212 B] Get:16 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble-backports/multiverse amd64 Components [212 B] Get:17 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [21.6 kB] Get:18 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Components [52.3 kB] Get:19 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Components [212 B] Get:20 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Components [208 B] Fetched 5650 kB in 1s (4128 kB/s) Reading package lists... Done ubuntu@ip-172-31-23-0:~\$</pre> sudo apt-get install -y ca-certificates curl gnupg <pre>ubuntu@ip-172-31-23-0:~\$ sudo apt-get install -y ca-certificates curl gnupg Reading package lists... Done Building dependency tree... Done Reading state information... Done ca-certificates is already the newest version (20240203). curl is already the newest version (8.5.0-2ubuntu10.6). gnupg is already the newest version (2.4.4-2ubuntu17.3). 0 upgraded, 0 newly installed, 0 to remove and 27 not upgraded. ubuntu@ip-172-31-23-0:~\$</pre>
5	Add docker official SPG key: <pre>sudo install -m 0755 -d /etc/apt/keyrings curl -fsSL https://download.docker.com/linux/ubuntu/gpg sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg sudo chmod a+r /etc/apt/keyrings/docker.gpg</pre>

	<pre> 0 upgraded, 0 newly installed, 0 to remove and 71 not upgraded. ubuntu@ip-172-31-23-0:~\$ sudo install -m 0755 -d /etc/apt/keyrings curl -fsSL https://download.docker.com/linux/ubuntu/gpg sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg sudo chmod a+r /etc/apt/keyrings/docker.gpg ubuntu@ip-172-31-23-0:~\$ </pre>
6	Add docker official repository: <pre> echo \ "deb [arch=\$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \ https://download.docker.com/linux/ubuntu \ \$(. /etc/os-release && echo "\$VERSION_CODENAME") stable" \ sudo tee /etc/apt/sources.list.d/docker.list > /dev/null </pre> <pre> ubuntu@ip-172-31-23-0:~\$ echo \ "deb [arch=\$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \ https://download.docker.com/linux/ubuntu \ \$(. /etc/os-release && echo "\$VERSION_CODENAME") stable" \ sudo tee /etc/apt/sources.list.d/docker.list > /dev/null ubuntu@ip-172-31-23-0:~\$ </pre> CloudShell Feedback © 2025, Amazon Web Services, Inc. or its affiliates
7	Update repositories: <pre> sudo apt-get update -y </pre> <pre> ubuntu@ip-172-31-23-0:~\$ sudo apt-get update -y Hit:1 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble InRelease Hit:2 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble-updates InRelease Hit:3 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble-backports InRelease Get:4 https://download.docker.com/linux/ubuntu noble InRelease [48.5 kB] Get:5 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB] Get:6 https://download.docker.com/linux/ubuntu noble/stable amd64 Packages [33.5 kB] Get:7 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [21.5 kB] Get:8 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Components [52.2 kB] Get:9 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Components [212 B] Get:10 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Components [212 B] Fetched 282 kB in 1s (458 kB/s) Reading package lists... Done ubuntu@ip-172-31-23-0:~\$ </pre>
8	Execute below command to install docker: sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin <pre> ubuntu@ip-172-31-23-0:~\$ sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin Reading package lists... Done Building dependency tree... Done Reading state information... Done The following additional packages will be installed: docker-ce-rootless-extras libslirp0 pigz slirp4netns Suggested packages: cgroupfs-mount cgroup-lite docker-model-plugin The following NEW packages will be installed: containerd.io docker-buildx-plugin docker-ce docker-ce-cli docker-ce-rootless-extras docker-compose-plugin libslirp0 pigz slirp4netns 0 upgraded, 9 newly installed, 0 to remove and 27 not upgraded. Need to get 105 MB of archives. After this operation, 437 MB of additional disk space will be used. Get:1 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 pigz amd64 2.8-1 [65.6 kB] Get:2 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble/main amd64 libslirp0 amd64 4.7.0-1ubuntu3 [63.8 kB] Get:3 https://download.docker.com/linux/ubuntu noble/stable amd64 containerd.io amd64 1.7.28-1~ubuntu.24.04~noble [31.9 MB] Get:4 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 slirp4netns amd64 1.2.1-1build2 [34.9 kB] Get:5 https://download.docker.com/linux/ubuntu noble/stable amd64 docker-ce-cli amd64 5:28.5.1-1~ubuntu.24.04~noble [16.5 MB] Get:6 https://download.docker.com/linux/ubuntu noble/stable amd64 docker-ce amd64 5:28.5.1-1~ubuntu.24.04~noble [19.7 MB] Get:7 https://download.docker.com/linux/ubuntu noble/stable amd64 docker-buildx-plugin amd64 0.29.1-1~ubuntu.24.04~noble [15.9 MB] </pre>

	<pre> Processing triggers for libc-bin (2.39-0ubuntu8.6) ... Scanning processes... Scanning linux images... Running kernel seems to be up-to-date. No services need to be restarted. No containers need to be restarted. No user sessions are running outdated binaries. No VM guests are running outdated hypervisor (qemu) binaries on this host. ubuntu@ip-172-31-23-0: ~\$ </pre>
9	Verify docker installation: docker --version <pre> No VM guests are running outdated hypervisor (qemu) binaries on this host. ubuntu@ip-172-31-23-0:~\$ docker --version Docker version 28.5.1, build e180ab8 ubuntu@ip-172-31-23-0:~\$ </pre>
10	Enable and Start Docker: Enable Docker to start on boot and start the Docker service: sudo systemctl enable docker sudo systemctl start docker <pre> Docker version 28.5.1, build e180ab8 ubuntu@ip-172-31-23-0:~\$ sudo systemctl enable docker Synchronizing state of docker.service with SysV service script with /usr/lib/systemd/systemd-sysv-install. Executing: /usr/lib/systemd/systemd-sysv-install enable docker ubuntu@ip-172-31-23-0:~\$ sudo systemctl start docker ubuntu@ip-172-31-23-0:~\$ </pre>
11	Run a test container "hello-world" to verify everything is working: docker run hello-world

```

bhargav_raghavendra@my-ubuntu-vm:~$ docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
c1ec31eb5944: Pull complete
Digest: sha256:5b3cc85e16e3058003c13b7821318369dad01dac3dbb877aac3c28182255c724
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/

```

Note: If you encounter a “**permission denied**” error while running Docker commands, follow the steps below to resolve it:

1. Check docker status : execute - `sudo systemctl status docker`
Docker status should be **Active**. If it's not active then **enable** and **start** docker.
2. Docker automatically creates a user group named “docker”.
To allow your current user to run Docker commands without using sudo, add the user to this group and refresh the group membership:

```

sudo usermod -aG docker $USER
newgrp docker

```

3. Now execute : `docker run hello-world`

12 Check docker image “hello-world” status:

docker images

```

ubuntu@ip-172-31-23-0:~$ docker images
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
hello-world   latest    1b44b5a3e06a   2 months ago   10.1kB
ubuntu@ip-172-31-23-0:~$

```


Activity 2: Create Docker File

2.1 Prerequisite

1. AWS account is ready to use.
2. Docker is installed.
3. Springboot application is ready for deployment.
4. DockerHub account (optional)
5. Basic knowledge of docker commands.
6. text file to write docker file instructions.

2.2 Scenario

Create a docker file to containerise SpringBoot application.

2.3 Steps

1	Clone git repository git clone https://gitlab.com/dmjadhav795/easyfly.git
2	Open any text editor Write docker file: Docker file will have specific instructions line by line.
3	Creating docker file to dockarise simple spring boot application: # Step 1: Use an official base image (OpenJDK 17) #for x86_64 architecture FROM openjdk:17-jdk-alpine #for ARM64 architecture #FROM --platform=linux/arm64 eclipse-temurin:17-jdk # Step 2: Copy your JAR file into the container COPY target/*.jar app.jar # Step 3: Expose the port your app runs on EXPOSE 8081 # Step 4: Run the application CMD ["java", "-jar", "/app.jar"]
4	Explanation about docker file: FROM: specifying JDK 17 image. COPY: copying application jar into /app container. EXPOSE: specify port number on which you app runs CMD: define commands to run application jar.

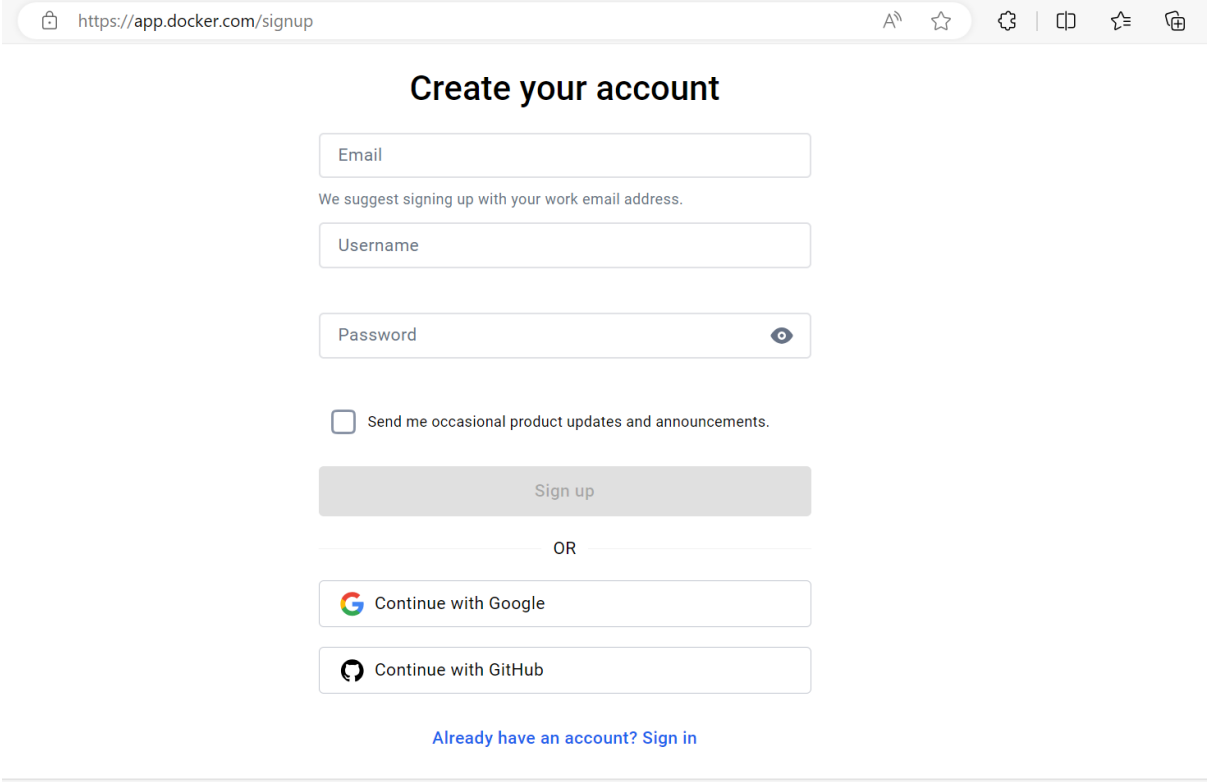
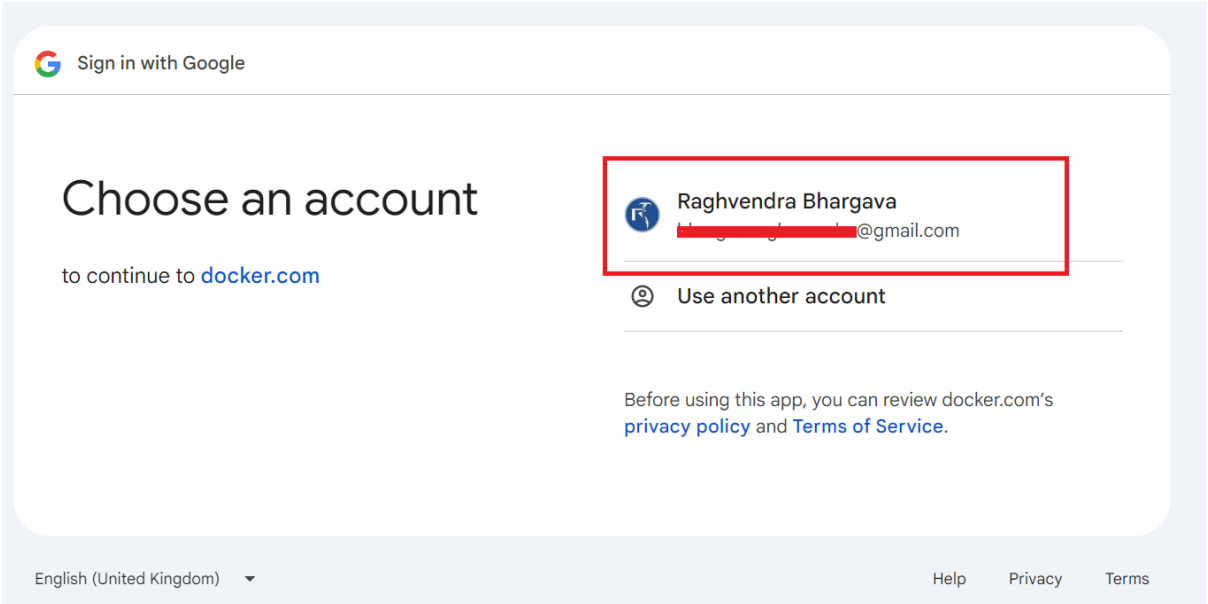
Activity 3: Create DockerHub Account

3.1 Prerequisite

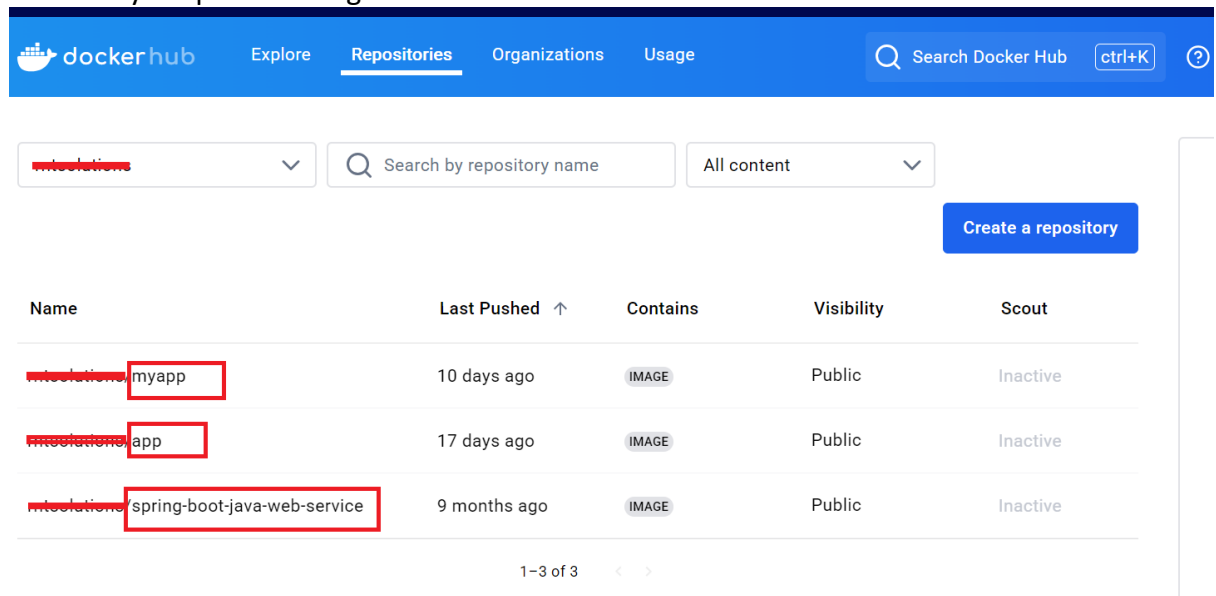
AWS cloud account is ready, Springboot application is ready, Docker file is ready.

3.2 Scenario

3.3 Steps

1	Browse https://app.docker.com/signup and signup 
2	Signup with your gmail account. 

- 3 Sign-in and browse <https://hub.docker.com/repositories/docker-id> your docker repository to check all your pushed images.



Activity 4: Create Docker Image

4.1 Prerequisite

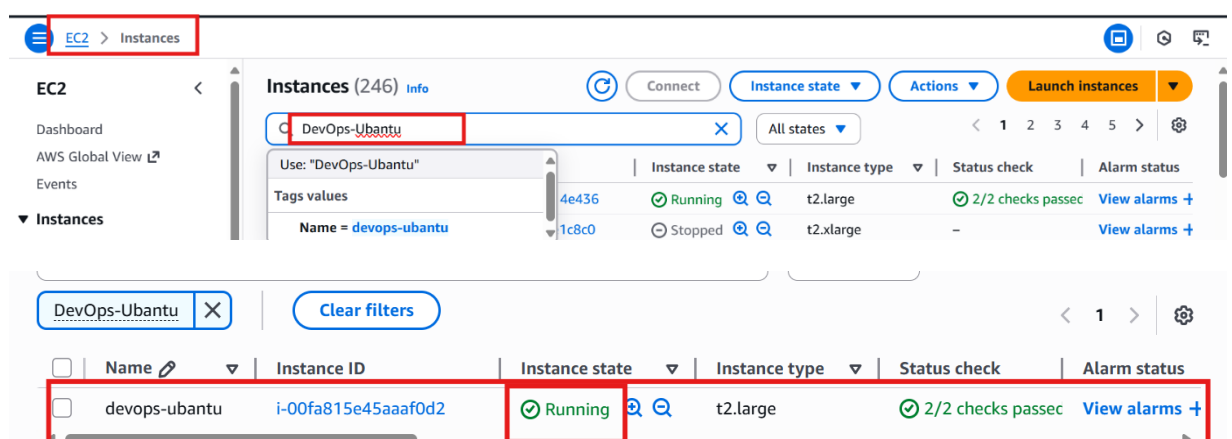
- AWS Cloud account is ready to use.
- Springboot application is ready for deployment.
- DockerHub account (optional)
- Basic knowledge of docker commands.

4.2 Scenario

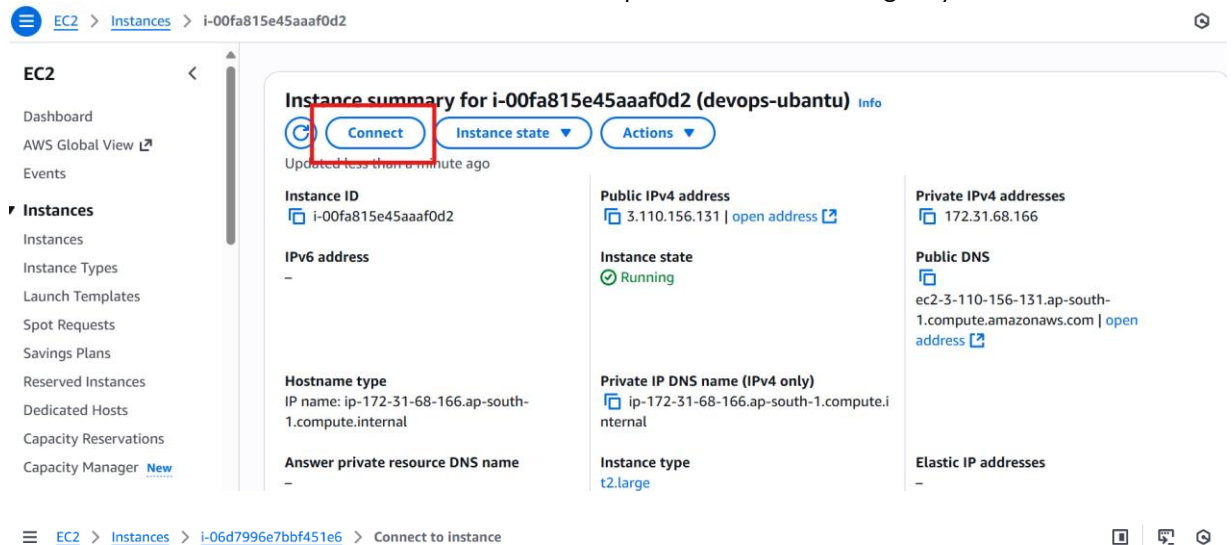
Launch AWS EC2 connect to create docker image to deploy spring boot application

4.3 Steps

- 1 Open AWS cloud console-> EC2-> instances-> Search instance by name or id



Click the **Connect** button to view the different options for connecting to your AWS instance.



EC2 > Instances > i-00fa815e45aaaf0d2

Instance summary for i-00fa815e45aaaf0d2 (devops-ubuntu) Info

Updated less than a minute ago

Connect Instance state Actions

Instance ID: i-00fa815e45aaaf0d2

Public IPv4 address: 3.110.156.131 | [open address](#)

Private IPv4 addresses: 172.31.68.166

Instance state: **Running**

Public DNS: ec2-3-110-156-131.ap-south-1.compute.amazonaws.com | [open address](#)

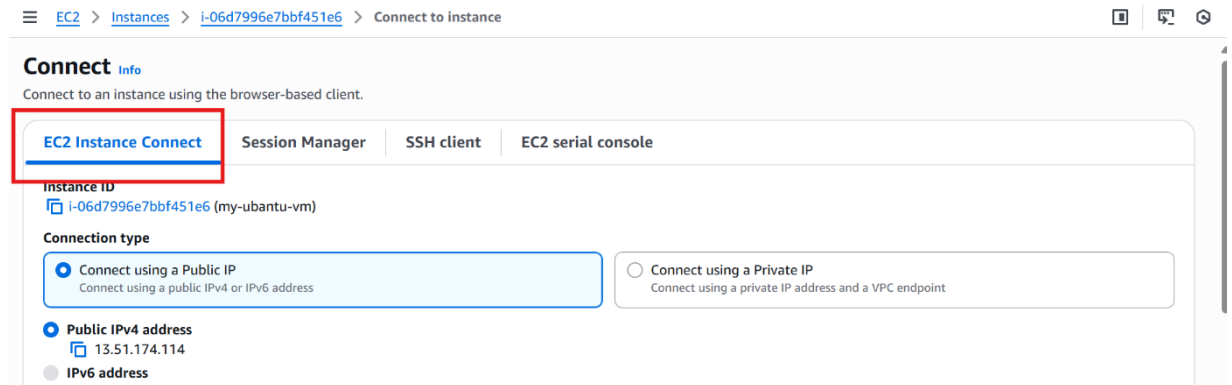
Private IP DNS name (IPv4 only): ip-172-31-68-166.ap-south-1.compute.internal

Instance type: t2.large

Hostname type: IP name: ip-172-31-68-166.ap-south-1.compute.internal

Answer private resource DNS name: -

Elastic IP addresses: -



EC2 > Instances > i-06d7996e7bbf451e6 > Connect to instance

Connect Info

Connect to an instance using the browser-based client.

EC2 Instance Connect Session Manager SSH client EC2 serial console

Instance ID: i-06d7996e7bbf451e6 (my-ubuntu-vm)

Connection type

☒ Connect using a Public IP
Connect using a public IPv4 or IPv6 address

☐ Connect using a Private IP
Connect using a private IP address and a VPC endpoint

☒ Public IPv4 address
13.51.174.114

☐ IPv6 address

Note: If you are using the Ubuntu Desktop (GUI) version, open a Terminal window and execute the commands listed below.

- 2 Clone SpringBoot project from gitlab/github by executing following command:
git clone https://github.com/your-git-hub-id/spring-boot.git

```
bhargav_raghavendra@my-ubuntu-vm:~$ git clone https://github.com/dadaramjadhav/spring-boot.git
Cloning into 'spring-boot'...
remote: Enumerating objects: 34, done.
remote: Counting objects: 100% (34/34), done.
remote: Compressing objects: 100% (21/21), done.
remote: Total 34 (delta 3), reused 27 (delta 2), pack-reused 0 (from 0)
Unpacking objects: 100% (34/34), 60.25 KiB | 1.77 MiB/s, done.
bhargav_raghavendra@my-ubuntu-vm:~$
```

- 3 "cd spring-boot" (browser project "spring-boot" folder)

```
bhargav_raghavendra@my-ubuntu-vm:~$ cd spring-boot
bhargav_raghavendra@my-ubuntu-vm:~/spring-boot$
```

- 4 Package spring boot app "mvn clean install" (package application as jar)

	<pre> bhargav_raghavendra@my-ubuntu-vm:~/spring-boot\$ mvn clean install [INFO] Scanning for projects... [INFO] [INFO] -----< com.accenture:spring-boot >----- [INFO] Building spring-boot 1.2 [INFO] -----[jar]----- [INFO] [INFO] --- maven-clean-plugin:3.2.0:clean (default-clean) @ spring-boot --- [INFO] [INFO] --- maven-resources-plugin:3.2.0:resources (default-resources) @ spring-boot --- [INFO] Using 'UTF-8' encoding to copy filtered resources. [INFO] Using 'UTF-8' encoding to copy filtered properties files. [INFO] Copying 1 resource [INFO] Copying 0 resource [INFO] [INFO] --- maven-compiler-plugin:3.10.1:compile (default-compile) @ spring-boot --- [INFO] Changes detected - recompiling the module! [INFO] Compiling 1 source file to /home/bhargav_raghavendra/spring-boot/target/classes [INFO] [INFO] --- maven-resources-plugin:3.2.0:testResources (default-testResources) @ spring-boot --- [INFO] Using 'UTF-8' encoding to copy filtered resources. [INFO] Using 'UTF-8' encoding to copy filtered properties files. [INFO] skip non existing resourceDirectory /home/bhargav_raghavendra/spring-boot/src/test/resources [INFO] [INFO] --- maven-compiler-plugin:3.10.1:testCompile (default-testCompile) @ spring-boot --- [INFO] Changes detected - recompiling the module! [INFO] Compiling 1 source file to /home/bhargav_raghavendra/spring-boot/target/test-classes [INFO] [INFO] --- maven-surefire-plugin:2.22.2:test (default-test) @ spring-boot --- [INFO] [INFO] ----- :: Spring Boot :: (v2.7.7) 2025-01-06 10:16:24.053 INFO 222246 --- [main] com.accenture.ApplicationTests : Starting A pplicationTests using Java 17.0.13 on my-ubuntu-vm with PID 222246 (started by bhargav_raghavendra in /home/bhar gav_raghavendra/spring-boot) 2025-01-06 10:16:24.056 INFO 222246 --- [main] com.accenture.ApplicationTests : No active profile set, falling back to 1 default profile: "default" 2025-01-06 10:16:26.932 INFO 222246 --- [main] com.accenture.ApplicationTests : Started Ap plicationTests in 3.424 seconds (JVM running for 5.591) [INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 5.271 s - in com.accenture.ApplicationTes ts [INFO] [INFO] Results: [INFO] [INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0 [INFO] [INFO] [INFO] --- maven-jar-plugin:3.2.2:jar (default-jar) @ spring-boot --- [INFO] Building jar: /home/bhargav_raghavendra/spring-boot/target/spring-boot-1.2.jar [INFO] [INFO] --- spring-boot-maven-plugin:2.7.7:repackage (repackage) @ spring-boot --- [INFO] Replacing main artifact with repackaged archive [INFO] [INFO] --- maven-install-plugin:2.5.2:install (default-install) @ spring-boot --- [INFO] Installing /home/bhargav_raghavendra/spring-boot/target/spring-boot-1.2.jar to /home/bhargav_raghavendra/ .m2/repository/com/accenture/spring-boot/1.2/spring-boot-1.2.jar [INFO] Installing /home/bhargav_raghavendra/spring-boot/pom.xml to /home/bhargav_raghavendra/.m2/repository/com/ accenture/spring-boot/1.2/spring-boot-1.2.pom [INFO] [INFO] BUILD SUCCESS [INFO] [INFO] Total time: 16.419 s [INFO] Finished at: 2025-01-06T10:16:30Z [INFO] </pre>
5	Browse target folder of spring boot app "ls target" <pre> bhargav_raghavendra@my-ubuntu-vm:~/spring-boot\$ ls target classes generated-test-sources maven-status spring-boot-1.2.jar.original test-classes generated-sources maven-archiver spring-boot-1.2.jar surefire-reports bhargav_raghavendra@my-ubuntu-vm:~/spring-boot\$ </pre>
6	Open editor to paste docker script by "vi Dockerfile" Save and exit from vi editor (press i for edit mode if required): Press Esc to exit insert mode.

	<pre> # Step 1: Use an official base image (OpenJDK 17) FROM openjdk:17-jdk-slim # Step 2: Set the working directory inside the container WORKDIR /app # Step 3: Copy your JAR file into the container COPY target/my-spring-app.jar app.jar # Step 4: Expose the port your app runs on EXPOSE 8080 # Step 5: Run the application CMD ["java", "-jar", "app.jar"] </pre> Type :wq and press Enter to save and exit from vi editor
7	Build image from docker script file by "docker build -t myapp ." ("myapp" is image name/tag in small case) <pre> bhargav_raghavendra@my-ubuntu-vm:~/spring-boot\$ docker build -t myapp . [+] Building 1.7s (9/9) FINISHED docker:default => [internal] load build definition from Dockerfile 0.0s => => transferring dockerfile: 394B 0.0s => [internal] load metadata for docker.io/library/openjdk:17-jdk-slim 0.3s => [auth] library/openjdk:pull token for registry-1.docker.io 0.0s => [internal] load .dockerignore 0.0s => => transferring context: 2B 0.0s => [1/3] FROM docker.io/library/openjdk:17-jdk-slim@sha256:aaa3b3cb27e3e520b8f116863d0580c438ed55ecfa0bc 0.0s => CACHED [2/3] WORKDIR /app 0.0s => [internal] load build context 0.4s => => transferring context: 17.65MB 0.4s => [3/3] COPY target/spring-boot-1.2.jar app.jar 0.6s => exporting to image 0.2s => => exporting layers 0.2s => => writing image sha256:7ddf936ea2f026c89cd3abf6595c57c21af6b2855abf2fc7f2da061d34ac2078 0.0s => => naming to docker.io/library/myapp 0.0s bhargav_raghavendra@my-ubuntu-vm:~/spring-boot\$ </pre>
8	Execute "docker images" command to check myapp image or list of docker images. <pre> bhargav_raghavendra@my-ubuntu-vm:~/spring-boot\$ docker images REPOSITORY TAG IMAGE ID CREATED SIZE myapp latest 7ddf936ea2f0 2 minutes ago 425MB rntolutions/app latest 0088df118fb7 6 days ago 425MB dadaramjadhav/cs latest 1b53dd69d1d5 5 months ago 345MB hello-world latest d2c94e258dcb 20 months ago 13.3kB openjdk latest 71260f256d19 23 months ago 470MB openjdk 11 47a932d998b7 2 years ago 654MB </pre>

Activity 5: Tag, List, Delete Image

5.1 Prerequisite

- AWS Cloud account is ready to use.
- Springboot application image is ready.
- DockerHub account (optional)
- Basic knowledge of docker commands.

5.2 Scenario

Tag, List and Delete existing docker image created in previous step.

5.3 Steps

1

To tag an existing Docker image named "**myapp**" with a new tag "**sbttag**", use the following command:

```
docker tag myapp:latest myapp:sbttag
```

tag image to docker hub, use following command:

```
docker tag myapp my-docker-hub-user/myapp:sbttag
```

where:

docker tag-> tag an existing image with new tag
myapp-> is existing docker image
myapp:sbttag-. New tag is applied

```
openjdk      11              47a932d998b7      2 years ago      654MB  
bhargav_paghavendra@my-ubuntu-vm:~$ docker tag myapp:latest myapp:sbttag
```

2

To verify new tagging, use following command to list images:

docker images

```
bhargav_paghavendra@my-ubuntu-vm:~$ docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
myapp	latest	7ddf936ea2f0	4 weeks ago	425MB
myapp	sbttag	7ddf936ea2f0	4 weeks ago	425MB
rntsolutions/myapp	latest	7ddf936ea2f0	4 weeks ago	425MB
rntsolutions/app	latest	0088df118fb7	4 weeks ago	425MB
mysql	latest	56a8c14e1404	3 months ago	603MB
mysql	8	3818a28b4a67	3 months ago	594MB
mysqlapp	latest	012163be757d	3 months ago	594MB
dadaramjadhav/cs	latest	1b53dd69d1d5	6 months ago	345MB
hello-world	latest	d2c94e258dcb	21 months ago	13.3kB
openjdk	latest	71260f256d19	24 months ago	470MB
openjdk	11	47a932d998b7	2 years ago	654MB

3

Remove image by image name:

If you have an image named **hello-world**, remove it with:

```
docker rmi hello-world
```

where hello-world is the image name.

```
bhargav_paghavendra@my-ubuntu-vm:~$ docker rmi hello-world  
Untagged: hello-world:latest  
Untagged: hello-world@sha256:5b3cc85e16e3058003c13b7821318369dad01dac3dbb877aac3c28182255c724  
Deleted: sha256:d2c94e258dcb3c5ac2798d32e1249e42ef01cba4841c2234249495f87264ac5a  
Deleted: sha256:ac28800ec8bb38d5c35b49d45a6ac4777544941199075dff8c4eb63e093aa81e
```

4 Remove image by using image tag:

If you have an image named myapp with the tag sbtag, remove it :

```
docker rmi myapp:sbtag
```

where “myapp” is image name and “sbtag” is tag name

```
binargav_raghavendra@my-ubuntu-vm:~$ docker rmi myapp:sbtag
Untagged: myapp:sbtag
```

5 To verify if image is removed by name and tag, list docker images:

docker images

```
binargav_raghavendra@my-ubuntu-vm:~$ docker images
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
myapp                latest             7ddf936ea2f0       4 weeks ago        425MB
rntsolutions/myapp  latest             7ddf936ea2f0       4 weeks ago        425MB
rntsolutions/app    latest             0088df118fb7       4 weeks ago        425MB
mysql               latest             56a8c14e1404       3 months ago       603MB
mysql               8                 3818a28b4a67       3 months ago       594MB
mysqlapp            latest             012163be757d       3 months ago       594MB
dadaramjadhav/cs    latest             1b53dd69d1d5       6 months ago       345MB
openjdk             latest             71260f256d19       24 months ago      470MB
openjdk             11                47a932d998b7       2 years ago        654MB
binargav_raghavendra@my-ubuntu-vm:~$
```

hello-world and myapp images is removed.

Activity 6: Push Image to dockerhub Registry

6.1 Prerequisite

- AWS Cloud account is ready to use.
- Springboot application image is ready.
- DockerHub account is ready.
- Basic knowledge of docker commands.

6.2 Scenario

Tag and push existing docker image to docker hub registry.

6.3 Steps

1 Login to docker hub if you are not logged in already using following command:

docker login

```
hhaugan_sagghavendra@my-ubuntu-vm:~$ docker login
Authenticating with existing credentials...
WARNING! Your password will be stored unencrypted in /home/hhaugan_sagghavendra/.docker/config.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credential-stores

Login Succeeded
hhaugan_sagghavendra@my-ubuntu-vm:~$
```

2 Before pushing, you need to tag your image with your Docker Hub username or repository name:

docker tag myapp:latest **my-docker-hub-username/myapp:v1.0**

where:

myapp is the docker image
v1.0 is tag name in hub repository

```
hhaugan_sagghavendra@my-ubuntu-vm:~$ docker tag myapp:latest rntsolutions/myapp:v1.0
```

3 List repo image using

docker images

```
hhaugan_sagghavendra@my-ubuntu-vm:~$ docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
myapp	latest	7ddf936ea2f0	4 weeks ago	425MB
rntsolutions/myapp	latest	7ddf936ea2f0	4 weeks ago	425MB
rntsolutions/myapp	v1.0	7ddf936ea2f0	4 weeks ago	425MB
rntsolutions/app	latest	0088d1181b7	5 weeks ago	425MB
mysql	latest	56a8c14e1404	3 months ago	603MB
mysql	8	3818a28b4a67	3 months ago	594MB
mysqlapp	latest	012163be757d	3 months ago	594MB
dadaramjadhav/cs	latest	1b53dd69d1d5	6 months ago	345MB
openjdk	latest	71260f256d19	24 months ago	470MB
openjdk	11	47a932d998b7	2 years ago	654MB

```
hhaugan_sagghavendra@my-ubuntu-vm:~$
```

4 Now, push the image to Docker Hub:

docker **push** my-docker-hub-username/myapp:v1.0

```
hhaugan_sagghavendra@my-ubuntu-vm:~$ docker push rntsolutions/myapp:v1.0
The push refers to repository [docker.io/rntsolutions/myapp]
bcba958b6f05: Layer already exists
d0d8bd31c400: Layer already exists
6be690267e47: Layer already exists
13a34b6fff78: Layer already exists
9clb6dd6c1e6: Layer already exists
v1.0: digest: sha256:5d35b572ff2647f510f583224d2413215177659c01d0d81139306c3c66c870aa size: 1371
hhaugan_sagghavendra@my-ubuntu-vm:~$
```

5 Verify pushed image on Docker Hub: <https://hub.docker.com/>

Go to Docker Hub and check if your image is uploaded.

intecolutions/myapp

Last pushed 1 minute ago • Repository size: 225.8 MB

Add a description

Add a category

General

Tags

Builds

Collaborators

Webhooks

Settings

Tags

This repository contains 2 tag(s).

Tag	OS	Type	Pulled	Pushed
 v1.0		Image	10 minutes ago	a minute ago
 latest		Image	10 minutes ago	10 minutes ago

[See all](#)

Last pushed 1 minute ago • Repository size: 225.8 MB

Add a category

Tags

Collaborators

Settings

This repository contains 2 tag(s).

OS

Type

Pulled

Pushed

■ v1.0

Image

10 minutes ago

a minute ago

- latest



Image

10 minutes ago

10 minutes ago

[See all](#)

7.1 Prerequisite

7.2 Scenario

7.3 Steps

```

root@my-ubuntu-vm:~# docker images

```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
myapp	latest	7ddcf936ea2f0	4 weeks ago	425MB
mysql	latest	56a8c14e1404	3 months ago	603MB
mysql	8	3818a28b4a67	3 months ago	594MB
mysqlapp	latest	012163be757d	3 months ago	594MB
openjdk	latest	71260f256d19	24 months ago	470MB
openjdk	11	47a932d998b7	2 years ago	654MB

2	Make sure you are logged-in to docker hub using “docker login” command. Then use “docker pull” command to pull docker image from registry by name:tag docker pull my-docker-hub-username/imageName <pre> bhargav_raghavendra@my-ubuntu-vm:~\$ docker pull rntsolutions/myapp Using default tag: latest latest: Pulling from rntsolutions/myapp Digest: sha256:5d35b572ff2647f510f583224d2413215177659c01d0d81139306c3c66c870aa Status: Downloaded newer image for rntsolutions/myapp:latest docker.io/rntsolutions/myapp:latest bhargav_raghavendra@my-ubuntu-vm:~\$ </pre>
3	Once the image is pulled/downloaded, check if it is available locally, use command: docker images <pre> bhargav_raghavendra@my-ubuntu-vm:~\$ docker images REPOSITORY TAG IMAGE ID CREATED SIZE mvapp latest 7ddf936ea2f0 4 weeks ago 425MB rntsolutions/myapp latest 7ddf936ea2f0 4 weeks ago 425MB mysql latest 56a8c14e1404 3 months ago 603MB mysql 8 3818a28b4a67 3 months ago 594MB mysqlapp latest 012163be757d 3 months ago 594MB openjdk latest 71260f256d19 24 months ago 470MB openjdk 11 47a932d998b7 2 years ago 654MB bhargav_raghavendra@my-ubuntu-vm:~\$ </pre>

Activity 8: Create Container

8.1 Prerequisite

- AWS Cloud account is ready to use.
- Springboot application image is ready.
- DockerHub account is ready.
- Basic knowledge of docker commands.

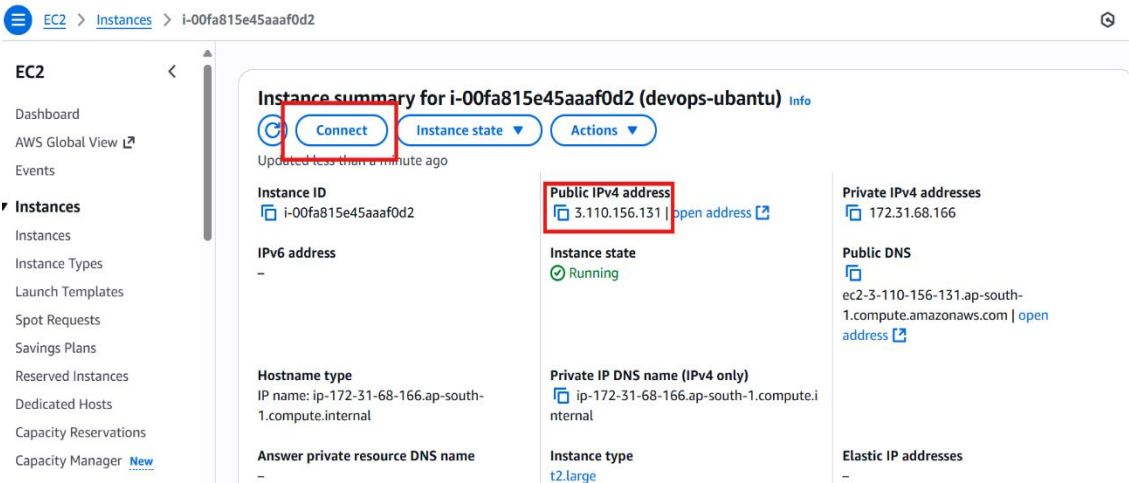
8.2 Scenario

Now create docker container from pulled docker image in previous step

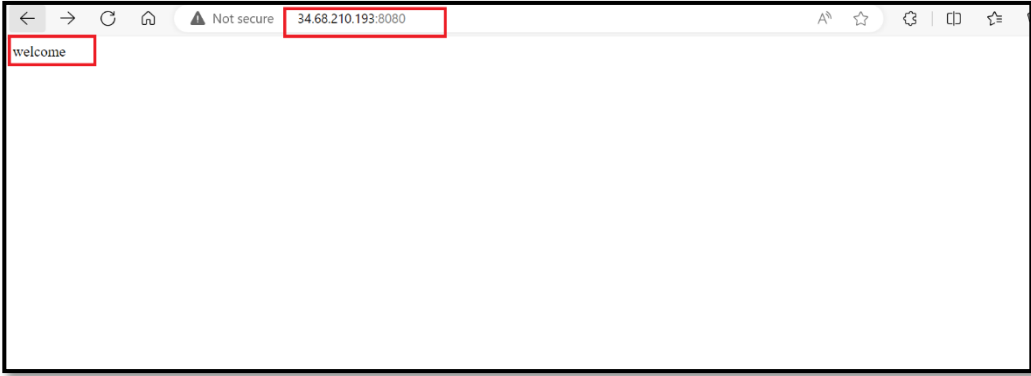
8.3 Steps

1	To run pulled image my-docker-username/myapp , use following command: docker run -d -p 8080:8080 my-docker-username/myapp:latest where -d runs container in detached mode -p 8080:8080 maps the container's port 8080 to your local machine. <pre> bhargav_raghavendra@my-ubuntu-vm:~\$ docker run -d -p 8080:8080 rntsolutions/myapp:latest d050ea76818198da5adc3dd3c6c43a10454d18dc5f18324a2c373865abeef4c/ bhargav_raghavendra@my-ubuntu-vm:~\$ </pre>
2	Now verify if container is running, using following command: “docker ps” <pre> bhargav_raghavendra@my-ubuntu-vm:~\$ docker ps CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS d050ea768181 rntsolutions/myapp:latest "java -jar app.jar" 36 seconds ago Up 35 seconds 0.0.0.0:8080->8080/tcp bhargav_raghavendra@my-ubuntu-vm:~\$ </pre>

3 Go to your EC2 instance and copy public IP address to access your application:



4 Now access your application using public ip address and port 8080
http://public-ip-address:port



Activity 9: Stop, Pause, List, Remove Container

9.1 Prerequisite

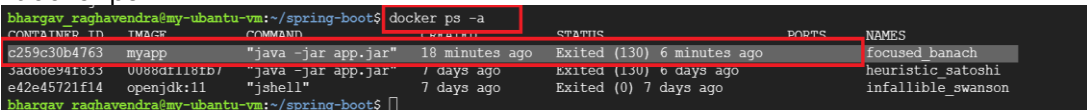
- AWS Cloud account is ready to use.
- Springboot application image is ready.
- DockerHub account is ready.
- Basic knowledge of docker commands.

9.2 Scenario

Now, manage the running container we created in the previous step by starting, stopping, pausing, listing, or removing it.

9.3 Steps

1 List the currently running containers using the following command:
"docker ps"



```

bhargav_raghavendra@my-ubuntu-vm:~/spring-boot$ docker ps -a
CONTAINER ID   IMAGE          COMMAND                  STATUS       PORTS          NAMES
c259c30b4763   myapp         "java -jar app.jar"     Exited (130) 6 minutes ago   focused_banach
3ad08e94f833   0088d1181b7   "java -jar app.jar"     Exited (130) 6 days ago      heuristic_satoshi
e42e45721f14   openjdk:11    "jshell"                 Exited (0) 7 days ago       infallible_swanson
bhargav_raghavendra@my-ubuntu-vm:~/spring-boot$

```

2	You can pause and resume a running container using the following commands: <pre>docker pause <container-id> / <container-name> -> pause a running container docker unpause <container-id> / <container-name> -> resume container</pre> <pre>bhargav_raghavendra@my-ubuntu-vm:~/spring-boot\$ docker pause 8806bd4181f4 8806bd4181f4 bhargav_raghavendra@my-ubuntu-vm:~/spring-boot\$ docker unpause 8806bd4181f4 8806bd4181f4 bhargav_raghavendra@my-ubuntu-vm:~/spring-boot\$</pre>
3	You can stop and start containers using the following commands: <pre>docker stop <container-id> → Stops a running container docker start <container-id> → Starts a stopped container</pre> <pre>bhargav_raghavendra@my-ubuntu-vm:~/spring-boot\$ docker stop 8806bd4181f4 8806bd4181f4 bhargav_raghavendra@my-ubuntu-vm:~/spring-boot\$ docker start 8806bd4181f4 8806bd4181f4 bhargav_raghavendra@my-ubuntu-vm:~/spring-boot\$</pre>
4	Now you can remove running container forcefully using following command: <pre>docker rm container-id -f</pre> where docker rm → Removes a container. Container-id-> Specifies the container id you want to remove. -f → Forces the removal of the container, even if it's running. -f flag will stop container first and then will remove it. <pre>bhargav_raghavendra@my-ubuntu-vm:~/spring-boot\$ docker rm 8806bd4181f4 -f 8806bd4181f4 bhargav_raghavendra@my-ubuntu-vm:~/spring-boot\$</pre>

Activity 10: Create MySQL Image

10.1 Prerequisite

- AWS Cloud account is ready to use.
- Basic knowledge of docker commands.

10.2 Scenario

Pull mysql image from registry, run it and execute mysql commands.

10.3 Steps

1	Pull MySQL 8 image from registry, use following command: <pre>docker pull mysql:8.0</pre>
---	--

	<pre>bhargav_raghavendra@my-ubuntu-vm:~\$ docker pull mysql:8.0 8.0: Pulling from library/mysql 4473ac30a868: Pull complete 5c3b09f953cc: Pull complete a69038e54211: Pull complete a0e40652ce5a: Pull complete cc5c2da6bb37: Pull complete a6258a15df35: Pull complete 483046b09121: Pull complete 8815fc443576: Pull complete b4af1f1b7267: Pull complete 2ef9232f1462: Pull complete 07884a158813: Pull complete Digest: sha256:f0657322a50b735bf284dac26e3e3d99e24810ea13d37ef6b7ca6cdd0d54fcc8 Status: Downloaded newer image for mysql:8.0 docker.io/library/mysql:8.0 bhargav_raghavendra@my-ubuntu-vm:~\$</pre>
2	Verify pulled mysql image in local repository, execute following command: docker images <pre>bhargav_raghavendra@my-ubuntu-vm:~\$ docker images REPOSITORY TAG IMAGE ID CREATED SIZE sb-myapp latest 2d5d7ab71ed5 3 days ago 425MB hello-world latest 74cc54e27dc4 2 weeks ago 10.1kB mysql 8.0 081ade299f2e 2 weeks ago 764MB bhargav_raghavendra@my-ubuntu-vm:~\$</pre>
3	Run mysql image to create mysql container using following command: <pre>docker run -d --name sb-mysql-db -e MYSQL_ROOT_PASSWORD=root -e MYSQL_DATABASE=mydb -e MYSQL_USER=myuser -e MYSQL_PASSWORD=root mysql:8.0</pre> where: docker run → Creates and starts a new mysql container. -d → Runs the container in detached mode (in the background). --name sb-mysql-db → Assigns the container a name (sb-mysql-db). -e MYSQL_ROOT_PASSWORD=root → Sets the root password for MySQL (root). -e MYSQL_DATABASE=mydb → Creates a database named mydb. -e MYSQL_USER=myuser → Creates a new user (myuser). -e MYSQL_PASSWORD=root → Sets the password for myuser (root). mysql:8.0 → Uses the MySQL version 8.0 Docker image. <pre>bhargav_raghavendra@my-ubuntu-vm:~\$ docker run -d --name sb-mysql-db -e MYSQL_ROOT_PASSWORD=root -e MYSQL_DATABASE=mydb -e MYSQL_USER=myuser -e MYSQL_PASSWORD=root mysql:8.0 cfe1d463e224bc0f30367ddef33bdbb52ab8d0e6548d28a7c8f24f4a8dd8d23c bhargav_raghavendra@my-ubuntu-vm:~\$</pre>
4	Verify if mysql container is running: docker ps

	<pre> bhargav_raghavendra@my-ubuntu-vm:~\$ docker run -d --name sb-mysql-db -e MYSQL_ROOT_PASSWORD=root -e MYSQL_DATABASE=mydb -e MYSQL_USER=myuser -e MYSQL_PASSWORD=root mysql:8.0 cfeld463e224bc0f30367ddef33bdbb52ab8d0e6548d28a7c8f24f4a8dd8d23c bhargav_raghavendra@my-ubuntu-vm:~\$ </pre>
5	Now let's verify and access the MySQL database inside a running container sb-mysql-db: Execute following command: <pre>docker exec -it sb-mysql-db mysql -u myuser -p</pre> docker exec → Runs a command inside an existing running container. -it → Allows interactive mode (-i for interactive input, -t for a pseudo-terminal). sb-mysql-db → The name of the running MySQL container. mysql → The MySQL client command to open the MySQL shell. -u myuser → Specifies the MySQL user (myuser) to log in. -p → Prompts for the password of myuser (you need to enter it manually). <pre> bhargav_raghavendra@my-ubuntu-vm:~\$ docker exec -it sb-mysql-db mysql -u myuser -p Enter password: Welcome to the MySQL monitor. Commands end with ; or \g. Your MySQL connection id is 8 Server version: 8.0.41 MySQL Community Server - GPL Copyright (c) 2000, 2025, Oracle and/or its affiliates. Oracle is a registered trademark of Oracle Corporation and/or its affiliates. Other names may be trademarks of their respective owners. Type 'help;' or '\h' for help. Type '\c' to clear the current input statement. mysql> </pre>
6	Now we are ready to access database and execute mysql commands Now list mysql databases and verify if mydb data is created! show databases; <pre> mysql> show databases; +-----+ Database +-----+ information_schema mydb performance_schema +-----+ 3 rows in set (0.01 sec) mysql> </pre>
7	Select mydb database to create table use mydb;

	<pre>mysql> use mydb; Database changed mysql> </pre>
8	Now create employee table in selected database mydb: <pre>CREATE TABLE employee (employeeId int, firstName varchar(255), lastName varchar(255), Address varchar(255), City varchar(255));</pre> <pre>mysql> CREATE TABLE employee (-> employeeId int, -> firstName varchar(255), -> lastName varchar(255), -> Address varchar(255), -> City varchar(255) ->); Query OK, 0 rows affected (0.05 sec) mysql> </pre>
9	Insert some records in employee table: <pre>insert into employee (employeeId, firstName, lastName, Address, City) values (1001, 'Myra', 'Bhargava', 'Indore', 'Indore'), (1002, 'Ravi', 'Paul', 'New Delhi', 'Delhi');</pre> <pre>mysql> insert into employee -> (employeeId, firstName, lastName, Address, City) -> values -> (1001, 'Myra', 'Bhargava', 'Indore', 'Indore'), -> (1002, 'Ravi', 'Sharma', 'Delhi', 'Delhi'); Query OK, 2 rows affected (0.02 sec) Records: 2 Duplicates: 0 Warnings: 0 mysql> mysql> </pre>
10	Now retrieve all records from employee table: <pre>select * from employee;</pre>

	<pre>mysql> select * from employee;</pre> <table><thead><tr><th>employeeId</th><th>firstName</th><th>lastName</th><th>Address</th><th>City</th></tr></thead><tbody><tr><td>1001</td><td>Myra</td><td>Bhargava</td><td>Indore</td><td>Indore</td></tr><tr><td>1002</td><td>Ravi</td><td>Sharma</td><td>Delhi</td><td>Delhi</td></tr></tbody></table> <pre>2 rows in set (0.00 sec) mysql> </pre>	employeeId	firstName	lastName	Address	City	1001	Myra	Bhargava	Indore	Indore	1002	Ravi	Sharma	Delhi	Delhi
employeeId	firstName	lastName	Address	City												
1001	Myra	Bhargava	Indore	Indore												
1002	Ravi	Sharma	Delhi	Delhi												
11	Let's remove the MySQL container and restart it to check if the mydb database is still accessible, along with its tables and data! <pre>bhargav_raghavendra@my-ubuntu-vm:~\$ docker rm -f sb-mysql-db sb-mysql-db bhargav_raghavendra@my-ubuntu-vm:~\$ </pre>															
12	Now, recreate the container using the following command: <pre>docker run -d --name sb-mysql-db -e MYSQL_ROOT_PASSWORD=root -e MYSQL_DATABASE=mydb -e MYSQL_USER=myuser -e MYSQL_PASSWORD=root mysql:8.0</pre> <pre>bhargav_raghavendra@my-ubuntu-vm:~\$ docker run -d --name sb-mysql-db -e MYSQL_ROOT_PASSWORD=root -e MYSQL_DATABASE=mydb -e MYSQL_USER=myuser -e MYSQL_PASSWORD=root mysql:8.0 cd08cddf24e1f8d6498de5c3eece1730c0c2af94fea66af011a118f2629a8575 bhargav_raghavendra@my-ubuntu-vm:~\$ docker ps</pre> <table><thead><tr><th>CONTAINER ID</th><th>IMAGE</th><th>COMMAND</th><th>CREATED</th><th>STATUS</th><th>PORTS</th><th>NAMES</th></tr></thead><tbody><tr><td>cd08cddf24e1</td><td>mysql:8.0</td><td>"docker-entrypoint.s..."</td><td>7 seconds ago</td><td>Up 6 seconds</td><td>3306/tcp, 33060/tcp</td><td>sb-mysq</td></tr></tbody></table> <pre>l-db bhargav_raghavendra@my-ubuntu-vm:~\$ </pre>	CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES	cd08cddf24e1	mysql:8.0	"docker-entrypoint.s..."	7 seconds ago	Up 6 seconds	3306/tcp, 33060/tcp	sb-mysq	
CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES										
cd08cddf24e1	mysql:8.0	"docker-entrypoint.s..."	7 seconds ago	Up 6 seconds	3306/tcp, 33060/tcp	sb-mysq										
13	Access the MySQL client inside the container: <pre>docker exec -it sb-mysql-db mysql -u myuser -p</pre> select mydb database: <pre>use mydb;</pre> Then, retrieve the records that were inserted earlier: <pre>select * from employee;</pre> <pre>mysql> use mydb; Database changed mysql> select * from employee; ERROR 1146 (42S02): Table 'mydb.employee' doesn't exist mysql> </pre> Refer above image, You will notice that the employee table is no longer accessible because when the MySQL container was stopped, all data in the mydb database was lost. This happens because the database was stored inside the container, which gets removed when the container is deleted. To prevent this data loss, we can use Docker volumes to persist the database files, ensuring that data remains intact even after restarting or recreating the container.															

Activity 11: Create and Attach Network

11.1 Prerequisite

- AWS Cloud account is ready to use.
- Basic knowledge of docker commands.
- Springboot application image is ready.
- MySQL image is ready to use

11.2 Scenario

Create a Spring Boot application container and a MySQL container on the same Docker network to enable seamless communication between them so containers can find each other by name.

11.3 Steps

1	Create custom network for the SpringBoot and MySQL containers to communicate. docker network create ltt_network docker network create - > is used to create custom network. ltt_network-> is the name of custom network <pre>bhargav_raghavendra@my-ubuntu-vm:~\$ docker network create ltt_network 4bf856191ef7fa104780e2fcf9d9792e486b6b4705bdb9e2fa2979cdb622ab61 bhargav_raghavendra@my-ubuntu-vm:~\$</pre>														
2	Now run MySQL container: <pre>docker run -d --name sb-mysql-db -e MYSQL_ROOT_PASSWORD=root -e MYSQL_DATABASE=mydb -e MYSQL_USER=myuser -e MYSQL_PASSWORD=root mysql:8.0</pre> <pre>bhargav_raghavendra@my-ubuntu-vm:~\$ docker run -d --name sb-mysql-db -e MYSQL_ROOT_PASSWORD=root -e MYSQL_DATABASE=mydb -e MYSQL_USER=myuser -e MYSQL_PASSWORD=root mysql:8.0 9aa649b5c43c337439299f627eedf7be2e4d50ab8f136e8acdd4d8d596860fab</pre>														
3	List mysql container: Docker ps <pre>bhargav_raghavendra@my-ubuntu-vm:~\$ docker ps</pre> <table><thead><tr><th>CONTAINER ID</th><th>IMAGE</th><th>COMMAND</th><th>CREATED</th><th>STATUS</th><th>PORTS</th><th>NAMES</th></tr></thead><tbody><tr><td>9aa649b5c43c</td><td>mysql:8.0</td><td>"docker-entrypoint.s..."</td><td>4 seconds ago</td><td>Up 4 seconds</td><td>3306/tcp, 33060/tcp</td><td>sb-mysq</td></tr></tbody></table> <pre>l-db bhargav_raghavendra@my-ubuntu-vm:~\$</pre>	CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES	9aa649b5c43c	mysql:8.0	"docker-entrypoint.s..."	4 seconds ago	Up 4 seconds	3306/tcp, 33060/tcp	sb-mysq
CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES									
9aa649b5c43c	mysql:8.0	"docker-entrypoint.s..."	4 seconds ago	Up 4 seconds	3306/tcp, 33060/tcp	sb-mysq									
4	Access the MySQL client inside the container: <pre>docker exec -it sb-mysql-db mysql -u myuser -p</pre> <pre>bhargav_raghavendra@my-ubuntu-vm:~\$ docker exec -it sb-mysql-db mysql -u myuser -p Enter password: Welcome to the MySQL monitor. Commands end with ; or \g. Your MySQL connection id is 10 Server version: 8.0.41 MySQL Community Server - GPL Copyright (c) 2000, 2025, Oracle and/or its affiliates. Oracle is a registered trademark of Oracle Corporation and/or its affiliates. Other names may be trademarks of their respective owners. Type 'help;' or '\h' for help. Type '\c' to clear the current input statement. mysql> ^C mysql></pre>														
5	Check mydb database which we have created: show databases:														

	<pre>mysql> show databases; +-----+ Database +-----+ information_schema mydb performance_schema +-----+ 3 rows in set (0.02 sec) mysql> </pre>																		
6	Now attach mysql container to the ltt_network: docker network connect ltt_network sb-mysql-db ltt_network-> is custom network sb-mysql-db-> is mysql container which we have created in previous step. <pre>bhargav_raghavendra@my-ubuntu-vm:~\$ docker network connect ltt_network sb-mysql-db bhargav_raghavendra@my-ubuntu-vm:~\$ </pre>																		
7	Now, run your Spring Boot container and attach it to the same network, use following command: docker run --name sb-myapp --network ltt_network -e SPRING_DATASOURCE_URL=jdbc:mysql://sb-mysql-db:3306/mydb -e SPRING_DATASOURCE_USERNAME=myuser -e SPRING_DATASOURCE_PASSWORD=root -d rntsolutions/myapp sb-myapp container connected to the network ltt_network. Passes the MySQL connection details as environment variables to the Spring Boot application. <pre>bhargav_raghavendra@my-ubuntu-vm:~\$ docker run --name sb-myapp --network ltt_network -e SPRING_DATASOURCE_URL=jdbc:mysql://sb-mysql-db:3306/mydb -e SPRING_DATASOURCE_USERNAME=myuser -e SPRING_DATASOURCE_PASSWORD=root -d rntsolutions/myapp 391b7da9f72234e86806fe2a2805cc0447498ff8794dc66ac/d94fbb77/d085c2</pre>																		
8	Now list running containers: docker ps <pre>bhargav_raghavendra@my-ubuntu-vm:~\$ docker ps</pre> <table><thead><tr><th>CONTAINER ID</th><th>IMAGE</th><th>COMMAND</th><th>CREATED</th><th>STATUS</th><th>PORTS</th></tr></thead><tbody><tr><td>391b7da9f722</td><td>rntsolutions/myapp</td><td>"java -jar app.jar"</td><td>8 seconds ago</td><td>Up 7 seconds</td><td>8080/tcp</td></tr><tr><td>9aa649b5c43c</td><td>mysql:8.0</td><td>"docker-entrypoint.s..."</td><td>14 minutes ago</td><td>Up 14 minutes</td><td>3306/tcp, 3306/tcp</td></tr></tbody></table> <pre>bhargav_raghavendra@my-ubuntu-vm:~\$ </pre>	CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	391b7da9f722	rntsolutions/myapp	"java -jar app.jar"	8 seconds ago	Up 7 seconds	8080/tcp	9aa649b5c43c	mysql:8.0	"docker-entrypoint.s..."	14 minutes ago	Up 14 minutes	3306/tcp, 3306/tcp
CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS														
391b7da9f722	rntsolutions/myapp	"java -jar app.jar"	8 seconds ago	Up 7 seconds	8080/tcp														
9aa649b5c43c	mysql:8.0	"docker-entrypoint.s..."	14 minutes ago	Up 14 minutes	3306/tcp, 3306/tcp														
9	Inspect the network to see the connected containers: docker network inspect ltt_network																		

	<pre> bhargav_raghavendra@my-ubuntu-vm:~\$ docker network inspect ltt_network [{ "Name": "ltt_network", "Id": "4bf856191ef7fa104780e2fcf9d9792e486b6b4705bdb9e2fa2979cdb622ab61", "Created": "2025-02-05T08:12:57.98034982Z", "Scope": "local", "Driver": "bridge", "EnableIPv6": false, "IPAM": { "Driver": "default", "Options": {}, "Config": [{ "Subnet": "172.18.0.0/16", "Gateway": "172.18.0.1" }] }, "Internal": false, "Attachable": false, "Ingress": false, "ConfigFrom": { "Network": "" }, "ConfigOnly": false, "Containers": { "391b7da9f72234e86806fe2a2805cc0447498ff8794dc66ac7d94fbb77d085c2": { "Name": "sb-myapp", "EndpointID": "6cee130d82545128ec893bb7083ff1eb2f412a3eaaa40b46a941558eefa5e518", "MacAddress": "02:42:ac:12:00:03", "IPv4Address": "172.18.0.3/16", "IPv6Address": "" } } }] </pre>
10	Verify connectivity between the Spring Boot container and the MySQL container by pinging MySQL from the Spring Boot container, ensuring both containers can communicate within the same network. docker exec -it sb-myapp ping sb-mysql-db <pre> bhargav_raghavendra@my-ubuntu-vm:~\$ docker exec -it sb-myapp ping sb-mysql-db PING sb-mysql-db (172.18.0.2) 56(84) bytes of data: 64 bytes from sb-mysql-db.ltt_network (172.18.0.2): icmp_seq=1 ttl=64 time=0.241 ms 64 bytes from sb-mysql-db.ltt_network (172.18.0.2): icmp_seq=2 ttl=64 time=0.092 ms 64 bytes from sb-mysql-db.ltt_network (172.18.0.2): icmp_seq=3 ttl=64 time=0.099 ms 64 bytes from sb-mysql-db.ltt_network (172.18.0.2): icmp_seq=4 ttl=64 time=0.081 ms 64 bytes from sb-mysql-db.ltt_network (172.18.0.2): icmp_seq=5 ttl=64 time=0.069 ms 64 bytes from sb-mysql-db.ltt_network (172.18.0.2): icmp_seq=6 ttl=64 time=0.067 ms 64 bytes from sb-mysql-db.ltt_network (172.18.0.2): icmp_seq=7 ttl=64 time=0.107 ms 64 bytes from sb-mysql-db.ltt_network (172.18.0.2): icmp_seq=8 ttl=64 time=0.079 ms 64 bytes from sb-mysql-db.ltt_network (172.18.0.2): icmp_seq=9 ttl=64 time=0.065 ms 64 bytes from sb-mysql-db.ltt_network (172.18.0.2): icmp_seq=10 ttl=64 time=0.102 ms 64 bytes from sb-mysql-db.ltt_network (172.18.0.2): icmp_seq=11 ttl=64 time=0.074 ms 64 bytes from sb-mysql-db.ltt_network (172.18.0.2): icmp_seq=12 ttl=64 time=0.091 ms 64 bytes from sb-mysql-db.ltt_network (172.18.0.2): icmp_seq=13 ttl=64 time=0.092 ms 64 bytes from sb-mysql-db.ltt_network (172.18.0.2): icmp_seq=14 ttl=64 time=0.079 ms 64 bytes from sb-mysql-db.ltt_network (172.18.0.2): icmp_seq=15 ttl=64 time=0.063 ms </pre>
11	if ping command is not working then first install ping inside springboot container, using following commands: 1. First, open a shell inside the container docker exec -it sb-myapp sh 2. Try installing ping based on your container OS (for ubuntu): apt update && apt install -y iputils-ping Exit and Try the ping Command Again docker exec -it sb-myapp ping sb-mysql-db

Activity 12: Create and Mount Volume

12.1 Prerequisite

- AWS Cloud account is ready to use.

- Basic knowledge of docker commands.
- MySQL image is ready to use

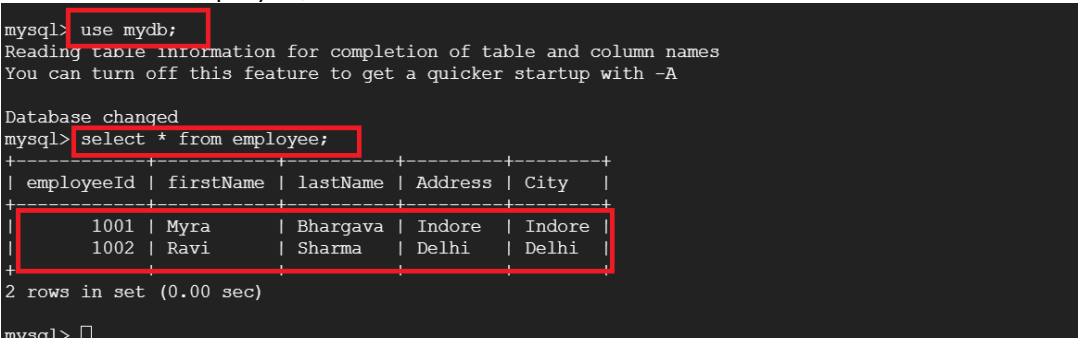
12.2 Scenario

Set up a MySQL container with a volume and mount it to store data externally, ensuring persistence beyond the container's lifecycle means data won't be lost when the container is deleted or restarted

12.3 Steps

1	Create docker volume: <code>docker volume create mysql-data</code> mysql-data-> The 'mysql-data' volume acts as a folder for storing data externally, separate from the container. <pre> bhargav_raghavendra@my-ubuntu-vm:~\$ docker volume create mysql-data mysql-data bhargav_raghavendra@my-ubuntu-vm:~\$ </pre>
2	Stop or remove MySQL container if its already running to avoid conflicts: <code>docker rm -f sb-mysql-db</code> <pre> bhargav_raghavendra@my-ubuntu-vm:~\$ docker rm -f sb-mysql-db sb-mysql-db </pre>
3	Now run mysql container aga in and attach it to the volume 'mysql-data' <code>docker run -d --name sb-mysql-db -e MYSQL_ROOT_PASSWORD=root -e MYSQL_DATABASE=mydb -e MYSQL_USER=myuser -e MYSQL_PASSWORD=root -v mysql-data:/var/lib/mysql mysql:8.0</code> -e MYSQL_ROOT_PASSWORD=root: Sets the MySQL root password. -v mysql-data:/var/lib/mysql: Maps the mysql-data volume to /var/lib/mysql inside the container. Since MySQL stores its database files in /var/lib/mysql by default, this ensures data persistence even after restarting/stop the container." <pre> bhargav_raghavendra@my-ubuntu-vm:~\$ docker run -d --name sb-mysql-db -e MYSQL_ROOT_PASSWORD=root -e MYSQL_DATABASE=mydb -e MYSQL_USER=myuser -e MYSQL_PASSWORD=root -v mysql-data:/var/lib/mysql mysql:8.0 fed2431915db7c5d21a16422ffb4bfe4812502473d7457e272d1711ac906d43 bhargav_raghavendra@my-ubuntu-vm:~\$ </pre>
4	Now verify if volume is mounted correctly: <code>docker inspect sb-mysql-db</code> <pre> bhargav_raghavendra@my-ubuntu-vm:~\$ docker inspect sb-mysql-db [{ "Id": "fed2431915db7c5d21a16422ffb4bfe4812502473d7457e272d1711ac906d43", "Created": "2025-02-05T11:36:04.179866636Z", "Path": "docker-entrypoint.sh", "Args": ["mysqld"], "State": { "Status": "running", "Running": true, "Paused": false, "Restarting": false, "OOMKilled": false, "Dead": false }, "HostConfig": { "Binds": ["mysql-data:/var/lib/mysql"], "ContainerIDFile": "", "LogConfig": { "Type": "json-file", "Config": {} }, "NetworkMode": "default", "PortBindings": {} } }] </pre>
5	Let's test the volume by storing some data. First, add data to the volume by running the following commands. Open the MySQL command line:

	<pre>docker exec -it sb-mysql-db mysql -u myuser -p bhargav_raghavendra@my-ubuntu-vm:~\$ docker exec -it sb-mysql-db mysql -u myuser -p Enter password: Welcome to the MySQL monitor. Commands end with ; or \g. Your MySQL connection id is 8 Server version: 8.0.41 MySQL Community Server - GPL Copyright (c) 2000, 2025, Oracle and/or its affiliates. Oracle is a registered trademark of Oracle Corporation and/or its affiliates. Other names may be trademarks of their respective owners. Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.</pre>
6	List databases: Show databases; <pre>mysql> show databases; +-----+ Database +-----+ information_schema mydb performance_schema +-----+ 3 rows in set (0.06 sec) mysql> </pre>
7	Select database mydb: use mydb; <pre>mysql> use mydb; Database changed mysql> </pre>
8	Create a table 'employee': CREATE TABLE employee (employeeId int, firstName varchar(255), lastName varchar(255), Address varchar(255), City varchar(255)); <pre>mysql> CREATE TABLE employee (-> employeeId int, -> firstName varchar(255), -> lastName varchar(255), -> Address varchar(255), -> City varchar(255) ->); Query OK, 0 rows affected (0.07 sec) mysql> </pre>
9	insert some records in the employee table: insert into employee (employeeId, firstName, lastName, Address, City) values (1001, 'Myra', 'Bhargava', 'Indore', 'Indore'), (1002, 'Ravi', 'Sharma', 'Delhi', 'Delhi'); <pre>mysql> insert into employee -> (employeeId, firstName, lastName, Address, City) -> values -> (1001, 'Myra', 'Bhargava', 'Indore', 'Indore'), -> (1002, 'Ravi', 'Sharma', 'Delhi', 'Delhi'); Query OK, 2 rows affected (0.07 sec) Records: 2 Duplicates: 0 Warnings: 0 'exit' from MySQL command line.</pre>
10	Now stop and remove mysql container: docker stop sb-mysql-db docker rm -f sb-mysql-db
11	Start new container with the same volume attached:

	docker run -d --name sb-mysql-db -e MYSQL_ROOT_PASSWORD=root -e MYSQL_DATABASE=mydb -e MYSQL_USER=myuser -e MYSQL_PASSWORD=root -v mysql-data:/var/lib/mysql mysql:8.0
12	Open mysql command line client: docker exec -it sb-mysql-db mysql -u myuser -p
13	Select mydb database: use mydb;
14	Check database to confirm if employee table and records are still there! Retrieve records from employee table: Select * from employee;  The mysql-data volume stores all data externally on the host system. Since volumes persist beyond container deletion or stopping, your MySQL data remains intact even if the container is removed or stopped."

Activity 13: Run Spring Boot and MySQL using Docker Compose

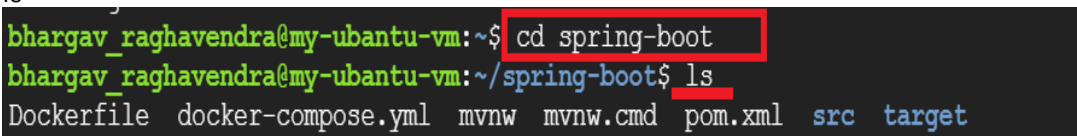
13.1 Prerequisite

- AWS Cloud account is ready to use.
- Springboot application image is ready.
- Mysql image is ready.
- Basic knowledge of docker commands.

13.2 Scenario

Use Docker Compose to launch both the Spring Boot application and MySQL containers simultaneously, streamlining the startup process instead of running them individually.

13.3 Steps

1	First, browse your project folder: cd spring-boot ls 
2	Create docker compose file : docker-compose.yml Find docker compose file in the given folder.
3.	Now open vi editor and create docker-compose.yml file in the project folder, using following command: vi docker-compose.yml

	<pre>unable to prepare context: path /home/bhargav_raghavendra/spring-boot/spring bhargav_raghavendra@my-ubuntu-vm:~/spring-boot\$ vi docker-compose.yml</pre>																														
4	Copy docker-compose.yml script and paste in the vi editor: Refer spring-boot image and mysql image specification in docker compose script. <pre>version: '3.8' # Docker Compose version services: sb-mysql-db: image: mysql:8.0 # MySQL image version container_name: sb-mysql-db # Name of the container restart: always # Automatically restart if it crashes environment: MYSQL_ROOT_PASSWORD: root # Root password for MySQL MYSQL_DATABASE: mydb # Database name MYSQL_USER: myuser # User for the database MYSQL_PASSWORD: root # Password for the user ports: - "3306:3306" # Map MySQL port to localhost volumes: - mysql-data:/var/lib/mysql # Persist MySQL data sb-myapp: build: context: . # Build the app from Dockerfile in the "app" folder container_name: sb-myapp # Name of the app container restart: always environment: SPRING_DATASOURCE_URL: jdbc:mysql://sb-mysql-db:3306/mydb SPRING_DATASOURCE_USERNAME: myuser SPRING_DATASOURCE_PASSWORD: root ports: - "8080:8080" # Map Spring Boot app port to localhost depends_on: - sb-mysql-db # Ensure MySQL starts before the app volumes: mysql-data: # Named volume for MySQL persistence</pre> "docker-compose.yml" 34L, 1312C 20,17 All Press 'i' to enter edit mode if needed. After making changes, press Esc to exit insert mode. Then, type :x and press Enter to save and exit the vi editor.																														
5	Now build and run docker compose: docker compose up --build or run containers in detached mode: docker compose up -d (detached mode)																														
6	List running containers: docker ps <pre>bhargav_raghavendra@my-ubuntu-vm:~/spring-boot\$ docker ps</pre> <table><thead><tr><th>CONTAINER ID</th><th>IMAGE</th><th>COMMAND</th><th>CREATED</th><th>STATUS</th><th>PORTS</th></tr></thead><tbody><tr><td>b80d8036146f</td><td>spring-boot-sb-myapp</td><td>"java -jar app.jar"</td><td>5 seconds ago</td><td>Up 4 seconds</td><td>8080/tcp, 0.0.0.0:</td></tr><tr><td>8081->8081/tcp, :::8081->8081/tcp</td><td>sb-myapp</td><td></td><td></td><td></td><td></td></tr><tr><td>634ba0433a2b</td><td>mysql:8.0</td><td>"docker-entrypoint.s..."</td><td>5 seconds ago</td><td>Up 4 seconds</td><td>0.0.0.0:3306->3306</td></tr><tr><td>/tcp, :::3306->3306/tcp, 33060/tcp</td><td>sb-mysql-db</td><td></td><td></td><td></td><td></td></tr></tbody></table> <pre>bhargav_raghavendra@my-ubuntu-vm:~/spring-boot\$</pre>	CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	b80d8036146f	spring-boot-sb-myapp	"java -jar app.jar"	5 seconds ago	Up 4 seconds	8080/tcp, 0.0.0.0:	8081->8081/tcp, :::8081->8081/tcp	sb-myapp					634ba0433a2b	mysql:8.0	"docker-entrypoint.s..."	5 seconds ago	Up 4 seconds	0.0.0.0:3306->3306	/tcp, :::3306->3306/tcp, 33060/tcp	sb-mysql-db				
CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS																										
b80d8036146f	spring-boot-sb-myapp	"java -jar app.jar"	5 seconds ago	Up 4 seconds	8080/tcp, 0.0.0.0:																										
8081->8081/tcp, :::8081->8081/tcp	sb-myapp																														
634ba0433a2b	mysql:8.0	"docker-entrypoint.s..."	5 seconds ago	Up 4 seconds	0.0.0.0:3306->3306																										
/tcp, :::3306->3306/tcp, 33060/tcp	sb-mysql-db																														
7	Access and verify if springboot application is working: http://public-ip-address:port 																														
8	Verify and access mysql container through mysql command line client: Open mysql command line client: docker exec -it sb-mysql-db mysql -u myuser -p																														


```
bhargav_raghavendra@my-ubuntu-vm:~/spring-boot$ docker exec -it sb-mysql-db mysql -u myuser -p
```

Enter password:

Welcome to the MySQL monitor. Commands end with ; or \g.

Your MySQL connection id is 8

Server version: 8.0.41 MySQL Community Server - GPL

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its affiliates. Other names may be trademarks of their respective owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

```
mysql> show databases;
```

```
+-----+
| Database |
+-----+
```

```
| information_schema |
+-----+
```

```
| mydb |
+-----+
```

```
| performance_schema |
+-----+
```

```
3 rows in set (0.01 sec)
```

```
mysql>
```

- 9 Access the Spring Boot application container shell and ping the MySQL container:
docker exec -it sb-myapp sh
and then ping mysql container:

ping sb-mysql-db

```
bhargav_raghavendra@my-ubuntu-vm:~/spring-boot$ docker exec -it sb-myapp sh
```

```
# ping sb-mysql-db
```

```
sh: 1: ping: not found
```

```
# apt update && apt install -y iputils-ping
```

```
Get:1 http://deb.debian.org/debian bullseye InRelease [116 kB]
```

```
Get:2 http://security.debian.org/debian-security bullseye-security InRelease [27.2 kB]
```

```
Get:3 http://deb.debian.org/debian bullseye-updates InRelease [44.1 kB]
```

```
Get:4 http://security.debian.org/debian-security bullseye-security/main amd64 Packages [344 kB]
```

```
Get:5 http://deb.debian.org/debian bullseye/main amd64 Packages [8066 kB]
```

```
Get:6 http://deb.debian.org/debian bullseye-updates/main amd64 Packages [18.8 kB]
```

```
Fetched 8616 kB in 2s (4440 kB/s)
```

```
Reading package lists... Done
```

```
Building dependency tree... Done
```

```
Reading state information... Done
```

```
44 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

```
Reading package lists... Done
```

```
Building dependency tree... Done
```

```
Reading state information... Done
```

```
The following additional packages will be installed:
```

```
libcap2 libcap2-bin libpam-cap
```

```
The following NEW packages will be installed:
```

```
iputils-ping libcap2 libcap2-bin libpam-cap
```

```
0 upgraded, 4 newly installed, 0 to remove and 44 not upgraded.
```

If ping command not found then install ping.